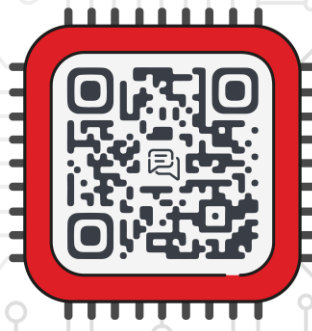




5

24

5400 sp

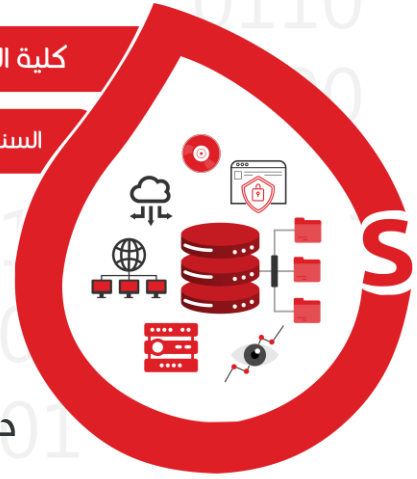
نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Query Processing - part1

د. مادلين عبود



RB Informatics; 23/10/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

هذه المحاضرة تشرح كيف يقوم نظام إدارة قواعد البيانات (DBMS) بمعالجة الاستعلامات، بدءاً من استعلام SQL مكتوب من المستخدم، مروراً بمراحل التحليل والترجمة وتقدير الكلفة، وصولاً إلى تنفيذ العمليات الأساسية على علاقة واحدة مثل الاختيار (Selection) والفرز (Sorting)، وذلك بهدف الحصول على النتائج المطلوبة بأقل كلفة ممكنة.

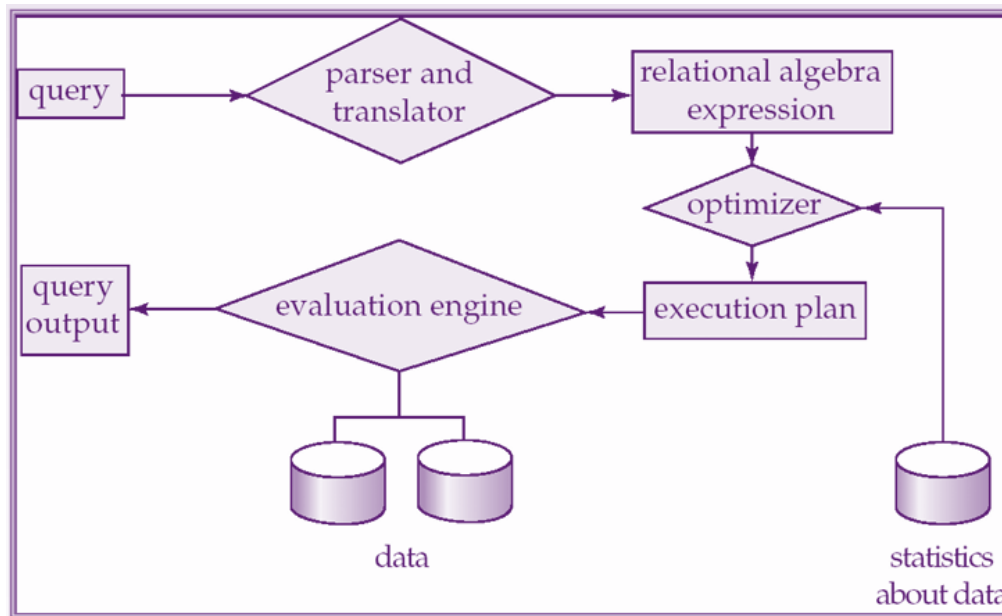
عادة:

المستخدم يكتب تعليمة مثلاً: SELECT

النظام لا ينفذها مباشرة.

بل يمر بعدة مراحل ذكية لاختيار أفضل طريقة تنفيذ.

ليكن لدينا المخطط التالي الذي يوضح مراحل مرور ال query وصولاً إلى ال output:



تفسير المخطط (سنشرح التفاصيل في الفقرة التالية):

Query: الاستعلام الذي يكتبه المستخدم.

Parser & Translator: التحقق والتحويل.

Optimizer: اختيار أفضل خطة.

Execution Plan (Tree): طريقة التنفيذ.

- Evaluation Engine: التنفيذ الفعلي.

- Query Output: النتيجة النهائية.

شرح الفكرة:

لنفرض أن لدينا استعلام (Query) مكتوب بلغة SQL Server. هذا الاستعلام لا يُنفَّذ مباشرة، وإنما يمر بعدة مراحل منظمة داخل نظام إدارة قواعد المعطيات (DBMS). أول مرحلة يمر بها الاستعلام هي:

1. مرحلة Parser and Translator:

1. Parser:

يطلق على هذه المرحلة أيضاً اسم:

Relation Verify and Syntax Check

وظيفة ال Parser:

1. التحقق من صحة الاستعلام من ناحية Syntax:

- أي التأكد من أن صيغة SQL صحيحة لغوياً.

مثال:

هل الكلمات مجوزة صحيحة؟

هل الأقواس مغلقة؟

هل ترتيب الجمل صحيح؟

في حال وجود أي خطأ لغوي (Syntax Error):

- يتم رفض الاستعلام مباشرة.

- ولا ننتقل لأي مرحلة لاحقة.

2. التحقق من قابلية الاستعلام للتنفيذ:

- التأكد من أن الجداول المذكورة موجودة.

- التأكد من أن الأعمدة (Attributes) موجودة ضمن هذه الجداول.

3. التحقق من عمليات الدمج (JOIN):

- إذا كان الاستعلام يحتوي على JOIN، يتم التأكد من:

1. وجود علاقة منطقية بين الجداول، أي وجود شرط ربط (مثل مفتاح أساسي مع مفتاح أجنبي).

لا يمكن تنفيذ استعلام على جدولين غير مرتبطين دون شرط ربط واضح.

لا تخف من البطء

خف من التوقف

2. Translator:

بعد أن يتأكد الـ Parser من أن الاستعلام صحيح وقابل للتنفيذ، ننتقل إلى مرحلة Translator.

وظيفة الـ Translator:

يقوم DBMS بتحويل الاستعلام المكتوب بلغة SQL إلى Relational Algebra Expression (التعبير بالجبر العلاقتي) وهي صيغة رياضية تعلمناها في مادة DB1 تمثل الاستعلام باستخدام:

- Selection (σ)
- Projection (π)
- Join ($\bowtie$)
- وغيرها

السبب:

- SQL لغة وصفية (ماذا تريد).
- جبر العلاقات لغة تنفيذية (كيف تنفذ).

2. مرحلة query optimization:

دور الـ Optimizer:

بعد تحويل الاستعلام إلى Relational Algebra Expression:

- يمكن تنفيذ نفس التعبير بعدة طرق مختلفة.
- هنا يأتي دور Query Optimizer

وظيفة الـ Optimizer:

اختيار أفضل طريقة تنفيذ (Best Execution Plan) اعتماداً على:
الإحصائيات - Statistics وهي معلومات مخزنة مسبقاً داخل النظام، مثل:

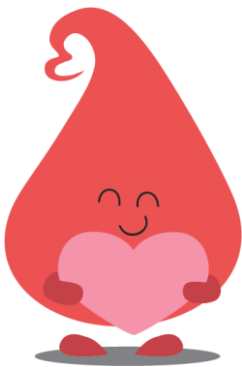
- حجم الجدول (Number of records).
- وجود فهارس (Indexes) على الأعمدة.
- نوع الفهارس وحجمها.
- طول السجل (Record length).
- توزيع القيم داخل الأعمدة.

الهدف:

تقليل كلفة التنفيذ خصوصاً (Disk I/O).

بعد أن يختار الـ Optimizer أنسب طريقة تنفيذ:

- ينتج لدينا ما يسمى Execution Plan، وهو:
- 1. تمثيل لطريقة التنفيذ.
- 2. يكون عادة على شكل Tree شجرة.



في هذه الشجرة:

- كل عملية (Selection, Join, Scan ...) تمثل بعقدة.
- كل نتيجة وسيطة تمثل بمربع.

3. مرحلة query evaluation:

بعد توليد Execution Plan:

- تنتقل الخطة إلى مرحلة Evaluation Engine
- حيث يتم:
 1. تنفيذ العمليات فعلياً.
 2. الوصول إلى قاعدة البيانات.
 3. جلب السجلات المطلوبة.



Query Output ↙

في النهاية:

- نحصل على Query Output
- يكون على شكل جدول
- يحتوي فقط على:
 1. السجلات
 2. والحقول المطلوبة في الاستعلام

ومنه نستنتج مراحل معالجة الاستعلام (Query Processing Steps):

تتضمن ثلاث مراحل أساسية:

1. التفسير والترجمة (Parsing and Translation):

- تحليل الاستعلام نحوياً (Syntax).
- التأكد من أنه صحيح لغوياً.
- تحويله إلى شكل داخلي Relational Algebra.

مثال:

SELECT * FROM account WHERE balance > 1000

يتم تحويله إلى:

 $\sigma(\text{balance} > 1000)(\text{account})$

التعب المؤقت أهون من عدم دائم

2. الاختزال (Query Optimization):

- أهم مرحلة.
- الهدف منها: اختيار **أقل كلفة تنفيذ**.
- يوجد أكثر من طريقة لتنفيذ نفس الاستعلام.

مثال:

- هل نستخدم index ؟
- هل نبدأ بالجدول A أم B ؟
- أي join method ؟

3. التقييم (Query Evaluation):

- تنفيذ الخطة المختارة فعلياً.
- الوصول إلى القرص.
- جلب السجلات.
- إنتاج النتيجة النهائية.

Why Query Optimization is important?لأن نفس الاستعلام يمكن تنفيذه بطرق مختلفة ب**فارق زمني هائل**.

مثال:

- تنفيذ بدون index يتم خلال ثوانٍ
- تنفيذ مع index يتم خلال أجزاء من الثانية

لذلك:

DBMS ذكي لا ينفذ أي خطة، بل يبحث عن الأفضل.

Measures of Query Cost

قياس كلفة تنفيذ الاستعلام

عند تنفيذ أي Query داخل نظام إدارة قواعد المعطيات، لا يكفي أن يكون الاستعلام صحيحاً، بل يجب أن نعرف:

- كم سيكلف تنفيذه؟
- ما المقصود بكلفة الاستعلام؟

بشكل عام، يتم قياس كلفة الاستعلام على أنها:
الزمن الكلي المنقضي (Total Elapsed Time) للحصول على نتيجة الاستعلام.

أي:

- من لحظة بدء التنفيذ.
- وحتى ظهور النتيجة النهائية للمستخدم.

العوامل المؤثرة في زمن التنفيذ:

زمن تنفيذ الاستعلام لا يعتمد على عامل واحد فقط، بل تتداخل عدة عوامل، أهمها:

1. Disk Accesses:

- عدد مرات الوصول إلى القرص.
- قراءة وكتابة البيانات.

2. CPU Time (سرعة وحدة المعالجة المركزية):

- الزمن اللازم لمعالجة البيانات.
- تنفيذ العمليات الحسابية والمنطقية.

3. الاتصال بالشبكة (في حال وجوده).

تعتبر عملية الوصول إلى القرص نسبياً سهلة التقدير

كيفية حساب كلفة الوصول إلى القرص:

أولاً يجب أن نعرف أن كلفة الاستعلام تقاس اعتماداً على ثلاثة عناصر رئيسية:

1. Number of seeks:

عدد مرات تحريك رأس القراءة على القرص.
كل Seek يعني:

- الانتقال إلى موقع جديد على القرص وهو مكلف زمنياً.

2. Number of blocks read:

عدد كتل البيانات (Blocks) التي يتم قراءتها من القرص.

3. Number of blocks written:

عدد كتل البيانات التي يتم كتابتها على القرص.

النجاح يحب الثبات لا الاندفاع

الصيغة العامة لحساب الكلفة:

يتم تقدير الكلفة باستخدام المعادلة التالية:

(Number of seeks × average-seek-cost)

+ (Number of blocks read × average-block-read-cost) +

(Number of blocks written × average-block-write-cost)

أي: كلفة التحريك + كلفة القراءة + كلفة الكتابة.

لماذا كلفة الكتابة أعلى من كلفة القراءة؟

عند كتابة Blocks على القرص:

يقوم النظام بقراءة البيانات مرة أخرى، للتأكد من أن عملية الكتابة تمت بنجاح.

لذلك:

$$\text{Cost}(\text{write block}) > \text{Cost}(\text{read block})$$

لتبسيط حساب الكلفة، يتم اعتماد ما يلي:

نستخدم فقط عدد الكتل المنقولة من وإلى القرص كمقياس للكلفة

أي:

- نهتم بعدد الكتل المقروءة.
- وعدد الكتل المكتوبة.

ولا نهتم بالتفريق بين:

- الخرج التسلسلي.
- والخرج العشوائي.

وذلك لأجل التبسيط فقط.

بالنسبة لكلفة CPU:

- في الأنظمة الحقيقية:

يتم أخذ كلفة المعالجة في ال CPU بالحسبان.

- لكن للتبسيط أيضاً:

1. نهمل الفرق في كلفة المعالجة بين الإدخال والإخراج.
2. ونهمل كلفة المعالجة في ال CPU.



تحرك الآن

فالتفكير وحده لا يغيّر شيئاً

وذلك بهدف:

جعل التحليل أبسط وأسهل للمقارنة بين الطرق المختلفة.
بناءً على ما سبق، يتم تقدير الكلفة اعتماداً على:

- عدد الكتل المقروءة.
- عدد الكتل المكتوبة.

فقط، دون:

- كلفة CPU أو تفاصيل إضافية.

تأثير الذاكرة الرئيسية (Buffer)

كلفة التنفيذ تعتمد أيضاً على حجم الذاكرة الرئيسية (Buffer):

- وجود ذاكرة كبيرة:
- يقلل الحاجة للوصول إلى القرص.
- لكن كمية الذاكرة المتاحة:
- تعتمد على نظام التشغيل الحالي وعدد العمليات الجارية.

لهذا السبب:

- نستخدم غالباً تقديرات أسوأ حالة Worst Case Estimation
- أي نفترض أن: كمية الذاكرة المتاحة هي الحد الأدنى لإنجاز العملية.

ملاحظة:

لا نضمن كلفة كتابة النتيجة النهائية على القرص ضمن حساب الكلفة لأنها:

- لا تؤثر بشكل جوهري على اختيار خطة التنفيذ.
- وغالباً تكون صغيرة مقارنة ببقية الكلفة.

تنفيذ تعليمة الاختيار (Selection Operation)

بعد أن تعرّفنا في الجزء السابق على **تقدير الكلفة** وأهمية حساب عدد الكتل المقروءة من القرص، ننتقل الآن إلى:

كيف ينفذ النظام تعليمة الاختيار (selection) σ؟ (أي كيف يجد مجموعة السجلات التي تحقق شرطاً معيناً)

تعليمية الاختيار تطبق باستخدام خوارزميات بحث مختلفة، ويختار بينها اعتماداً على:

- وجود فهرس أو عدمه.
- نوع شرط الاختيار.
- الكلفة المتوقعة.

فيما يلي سناقش الحالات المختلفة وسنستخدم المثال الموحد التالي:
لنفرض لدينا علاقة (جدول):

Account (branch_name, account_no, balance)

account_no	branch_name	balance
101	Damascus	500
102	Aleppo	300
103	Damascus	900
104	Homs	700
105	Damascus	300
106	Aleppo	1200

افتراضات ثابتة (تذكرها سنستخدم عليها في التطبيق في الأسفل):

- عدد السجلات = $nr = 6$.
- عدد الكتل = $br = 3$.
- عدد السجلات في الكتلة = $fr = 2$.
- يوجد فهرس أساسي على account_no.
- يوجد فهرس ثانوي على branch_name.
- يوجد فهرس ثانوي على balance.

سنستخدم نفس الجدول في كل الخوارزميات (تكملة الفقرة بعد التذكرة الاربيسيزية).

تذكرة اربيسيزية

تذكرة بالفهرس الأساسي والثانوي وفائدة تخص المفتاح المرشح (ستساعدنا بعد الشعور بالضياع أثناء شرح الخوارزميات في الأسفل):

الفهرس الأساسي (Primary Index):

- يُبنى على الصفة التي **الملف مرتب فعلياً حسبها**.
- يوجد **فهرس أساسي واحد** فقط لكل ملف.

في مثالنا:

يوجد فهرس أساسي على account_no

هذا يعني:

- ملف Account مفروز حسب account_no.
- السجلات ذات القيم المتقاربة مخزنة في كتل متجاورة.
- البحث بالمقارنة ($\geq$, $\leq$) فعال جداً.

نتيجة مهمة:

- السجلات المتطابقة غالباً في كتل متتالية.
- عدد عمليات القراءة أقل.

الفهرس الثانوي (Secondary Index):

- يُبنى على صفة لا يفرض ترتيب الملف.
- يمكن أن يوجد عدة فهرس ثانوية على نفس الملف.

في مثالنا:

يوجد فهرس ثانوي على branch_name.

يوجد فهرس ثانوي على balance.

هذا يعني:

- الملف غير مفروز حسب branch_name أو balance.
- الفهرس يحتوي:

1. قيمة الصفة.

2. قائمة مؤشرات (Pointers) للسجلات.

نتيجة مهمة:

السجلات المتطابقة قد تكون:

- موزعة على كتل مختلفة.
- كل سجل يحتاج قراءة مستقلة.

كيف يبدو الفهرس الثانوي على branch-name؟

الفهرس لا يخزن السجلات، بل يخزن:

القيمة + مؤشرات (Pointers) إلى السجلات:

branch_name index

Aleppo → [ptr → record 102]

Damascus → [ptr → record 101, ptr → record 103, ptr → record 105]

Homs → [ptr → record 104]

افعل الصعب اليوم، تعيش السهل غداً

كيف يبدو الفهرس الثانوي على account-no (نظرياً)؟

سيفيد هذا التصور في خوارزمية A5 (الحالة 1) في الأسفل
لو افترضنا (افتراضاً فقط) وجود فهرس ثانوي على account_no (وهو المفتاح المرشح) سيبدو هكذا:

```
account_no index (secondary)
-----
101 → [ptr → record 101]
102 → [ptr → record 102]
103 → [ptr → record 103]
...
```

لكل قيمة مؤشر واحد فقط ← لأنه مفتاح مرشح ← لا يوجد تكرار.

إضافة للتوضيح: ما هو Candidate Key (مفتاح مرشح)؟

صفة (أو مجموعة صفات) يمكنها تمييز كل سجل في الجدول بشكل فريد، أي:

- لا تتكرر
- لا تحتوي NULL

بمعنى آخر هو:

- أي صفة تصلح أن تكون مفتاحاً أساسياً.

نهاية التذكرة الاربيسيزية

الآن نعود لنبدأ بالخوارزميات:

1. البحث من دون استخدام فهرس (File Scan):

⚡ A1: البحث الخطي (Linear Search / File Scan)

في هذه الخوارزمية:

- يتم مسح كامل الملف.
- كل كتلة تقرأ من القرص.
- وكل سجل يُختبر لمعرفة إن كان يحقق شرط الاختيار.

متى نستخدم؟

- يمكن استخدامها دائماً
- بغض النظر عن:
 1. نوع الشرط.
 2. ترتيب السجلات.
 3. وجود فهرس من عدمه.

الرموز المستخدمة:

▪ Br: عدد كتل القرص التي تحتوي سجلات العلاقة r (التي تجري عملية الاختيار عليها).

الكلفة المتوقعة:

- في الحالة العامة:
نحتاج لقراءة جميع الكتل ← $cost = br$
- إذا كان شرط الاختيار على مفتاح:
يمكن التوقف عند إيجاد السجل ← الكلفة التقريبية $= br / 2$

تطبيق على المثال:

```
SELECT * FROM Account WHERE branch_name = 'Damascus';
```

ماذا يفعل النظام؟

يقرأ الكتلة الأولى: يفحص سجلين.

يقرأ الكتلة الثانية: يفحص سجلين.

يقرأ الكتلة الثالثة: يفحص سجلين.

يختبر كل سجل هل $branch_name = \text{Damascuse}$

النتيجة:

101, 103, 105

الكلفة:

قرأنا كل الكتل ← $cost = br = 3$

حالة خاصة (شرط على مفتاح):

```
SELECT * FROM Account WHERE account_no = 104;
```

بما أن $account_no$ مفتاح

قد نعثر عليه في منتصف الملف

← $cost \approx br / 2 = 1.5 \approx 2$

كل مرة تختار فيها الانصباط

تقترب خطوة من الحياة التي تريدها

A2: البحث الثنائي (Binary Search)

تستخدم هذه الخوارزمية فقط بشروط محددة:

شروط التطبيق:

1. شرط الاختيار هو مساواة.
2. الملف المفروز (ordered) حسب الصفة المستخدمة في الشرط.
3. كتل الملف مخزنة بشكل متتال

آلية العمل:

- نستخدم البحث الثنائي على مستوى الكتل، للوصول إلى أول سجل يحقق الشرط
- ثم نقرأ الكتل التي تحتوي السجلات المتطابقة
- كلفة الوصول لأول سجل: $|\log_2(br)|$.
- زائد عدد الكتل التي تحتوي على السجلات المتطابقة: $|sc(A,r) / fr|$

حيث:

- $sc(A,r)$ عدد السجلات التي تحقق الشرط.
- fr عدد السجلات في الكتلة.

الكلفة النهائية التقديرية:

$$|\log_2(br)| + |sc(A,r) / fr| - 1$$

تطبيق على المثال:

```
SELECT * FROM Account WHERE account_no = 105;
```

الشروط متحققة:

- مساواة.
- الملف مفروز حسب $account_no$ (وجود فهرس أساسي على رقم الحساب) ← الملف مرتب حسب رقم الحساب).
- الكتل متتالية.

ماذا يحدث؟

1. نبحث تلقائياً بين الكتل
2. ثم نصل لأول سجل يحقق الشرط
3. ثم نقرأ الكتلة التي تحتويه فقط الحساب:

- الوصول لأول سجل: $[\log_2(3)] = 2$

- عدد السجلات المطابقة = 1

$$\text{عدد الكتل} = [1 / 2] = 1$$

$$\text{الكلفة} \leftarrow 2 + 1 - 2 = 2$$

2. الاختيار باستخدام الفهارس (Index Scan):

ننتقل الآن إلى خوارزميات تستفيد من وجود Index:

↪ A3: فهرس أساسي على مفتاح مرشح (Primary Index on Candidate Key)

الشرط:

- شرط مساواة.
- الصفة هي مفتاح مرشح.
- النتيجة: سجل واحد فقط.

الكلفة:

- $Cost = HT_i + 1$
- حيث: HT_i هي كلفة الوصول إلى السجل في الفهرس.

تطبيق على المثال:

```
SELECT * FROM Account WHERE account_no = 103;
```

ماذا يحدث؟

1. نبحث في الفهرس

2. نحصل مباشرة على عنوان السجل

3. نقرأ كتلة واحدة فقط.

النتيجة:

$$Cost = HT_i + 1$$

(الوصول للفهرس + قراءة الكتلة)

↪ A4: فهرس أساسي على صفة ليست مفتاحاً

النتيجة:

- عدة سجلات مخزنة في كتل متتالية.

الكلفة:

$$Cost = HT_i + \lceil sc(A, r) / fr \rceil$$

تطبيق على المثال:

```
SELECT * FROM Account WHERE branch_name = 'Damascus';
```

ماذا يحدث؟

1. نصل لأول سجل في الفهرس

2. سجلات Damascus مخزنة متجاورة

3. نقرأ عدة كتل متتالية

عدد السجلات المطابقة: $sc = 3$

عدد الكتل: $\lceil 3 / 2 \rceil = 2$

الكلفة: $Cost = HT_i + 2$

A5: فهرس ثانوي (Secondary Index)

يوجد حالتين:

1. الصفة مفتاح مرشح (افترض أن لدينا فهرساً ثانوياً مبنياً على صفة هي مفتاح مرشح):

النتيجة (أي نتيجة الاستعلام):

▪ سجل واحد.

الكلفة:

$$\text{Cost} = \text{HTi} + 1$$

تطبيق على المثال:

SELECT * FROM Account WHERE account_no = 10;

1. الوصول إلى الفهرس:

account_no = 101

نبحث عن:

← نجد مدخلاً واحداً فقط.

← نحصل على Pointer واحد (انظر إلى شكل الفهرس التخيلي في التذكرة الأربيسيزية في الأعلى)

الكلفة:

▪ HTi.

2. جلب السجل من الملف:

نستخدم المؤشر (الذي سيكون مؤشراً على سجل واحد فقط لأن الفهرس مبنى على مفتاح== كل سجل له قيمة مفتاح فريدة).

← نقرأ سجلاً واحداً فقط.

الكلفة:

▪ + 1

إذا الكلفة النهائية:

$$\text{Cost} = \text{HTi} + 1$$

▪ أي هي كلفة الوصول إلى المؤشر المطلوب + قراءة السجل الذي يؤشر إليه (وهو سجل واحد).

ملاحظة مهمة:

رغم أننا ذكرنا أن account_no لديه فهرس أساسي،
نفترض هنا وجود فهرس ثانوي عليه فقط لأغراض الشرح والمقارنة،
وذلك لتوضيح أن كون الصفة مفتاحاً مرشحاً يؤدي دائماً إلى استرجاع سجل واحد فقط،
حتى عند استخدام فهرس ثانوي.

2. الصفة ليست مفتاحاً:

النتيجة:

- عدة سجلات
- قد تكون كل تسجيلة في كتلة مختلفة

الكلفة:

- $Cost = HT_i + sc(A, r)$
- قد تكون الكلفة عالية جداً

تطبيق على المثال:

```
SELECT * FROM Account WHERE branch_name = 'Damascus';
```

ماذا يحدث؟

1. الفهرس يعطي مؤشرات:

101 ←

103 ←

105 ←

2. كل سجل قد يكون في كتلة مختلفة؟

الكلفة:

$$Cost = HT_i + sc = HT_i + 3$$

قد تكون أعلى من المسح الخطي.

3. الاختيار مع المقارنات (<,>):

A6: فهرس أساسي + مقارنة

 $\sigma A \geq V$

- نستخدم الفهرس للوصول لأول سجل $V \geq$ (أول سجل فيه قيمة أكبر أو تساوي V أي أول سجل يحقق الشرط).
- ثم مسح تسلسلي.

 $\sigma A \leq V$

- مسح تسلسلي فقط.
- لا نستخدم الفهرس.

الكلفة:

$$Cost = HT_i + |c| / fr$$

حيث: c تمثل عدد التسجيلات المحققة للشرط

تطبيق على المثال:

1. في حالة أكبر أو يساوي:

```
SELECT * FROM Account WHERE account_no >= 103;
```

ماذا يحدث؟

1. نستخدم الفهرس للوصول لأول $account_no \geq 10$ 2. ثم نكمل ب مسح تسلسلي

النتيجة:

103, 104, 105, 106

الكلفة:

$$Cost = HT_i + \lceil 4 / 2 \rceil = HT_i + 2$$

2. في حالة أصغر أو يساوي:

```
SELECT * FROM Account WHERE account_no <= 103;
```

ماذا يحدث؟

1. لا نستخدم الفهرس 2. مسح تسلسلي من البداية

A7: فهرس ثانوي + مقارنة

- نبحث في أوراق الفهرس عن المؤشرات.
- ثم نصل إلى السجلات واحدة واحدة.
- قد تكون الكلفة أعلى من المسح الخطي.

تطبيق على المثال:

```
SELECT * FROM Account WHERE balance >= 700;
```

ماذا يحدث؟

1. نبحث في أوراق الفهرس 2. نحصل على مؤشرات 3. نقرأ كل سجل منفصلاً

لذلك الكلفة يمكن أن تكون أعلى من المسح الخطي.

4. تنفيذ الاختيارات المعقدة:

Conjunction (AND) $(\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r))$

لحساب عدد السجلات المتوقعة:

- نفترض استقلال الشروط.
- S_i عدد السجلات التي تحقق الشرط θ_i
- N_r عدد سجلات العلاقة

للتوضيح: n تمثل عدد الشروط

العدد المتوقع:

$$Sc(A, r) = nr * (s1 * s2 * ... * sn) / nr^n$$

تطبيق على المثال:

```
SELECT * FROM Account
WHERE branch_name = 'Damascus'
AND balance >= 500;
```

حساب الحجم المتوقع:

s1 = 3 (Damascus)

s2 = 3 (balance >= 500)

nr = 6

sc = 6 * (3 * 3) / 6² = 1.5 ≈ 2

⚡ A8: استخدام فهرس واحد

3. نخزن النتيجة في buffer
4. نختبر باقي الشروط

1. نختار الشرط الأقل كلفة بتطبيق إحدى الخوارزميات من A1 إلى A7
2. ننفذه أولاً

من أجل نفس الاستعلام السابق:

3. نخزن النتائج في buffer
4. نختبر balance

1. نختار الأرخص مثل (branch_name)
2. ننفذ

⚡ A9: فهرس مركب (Multiple-key Index)

- ثم نتحقق من باقي الشروط

- إذا توفر فهرس على مجموعة صفات
 - نبحت مباشرة
- من أجل الاستعلام السابق نفسه:

- لو وجد فهرس على: (branch_name, balance)
- نبحت مباشرة ← أقل كلفة.

⚡ A10: تقاطع المؤشرات

1. نستخدم فهرساً لكل شرط.
2. نأخذ تقاطع المؤشرات
3. ثم نصل إلى السجلات في الملف الأساسي.

من أجل نفس الاستعلام السابق لدينا:

- فهرس على branch_name
- فهرس على balance
- نأخذ التقاطع: $\{101,103,105\} \cap \{103,104,106\} = \{103\}$

5. Not & OR :

Disjunction (OR) : A11 ↙

- إذا توفرت فهارس لكل الشروط
- ← نأخذ اتحاد المؤشرات.
- والا:
- ← نستخدم المسح الخطي.

مثال:

branch_name = 'Damascus' OR balance >= 1000

Negation (NOT) ↙

- غالباً:
- ← نستخدم المسح الخطي.
- إذا كانت السجلات التي تحقق الشرط قليلة:
- ← نستخدم الفهرس ثم نكمل من الملف (Index Seek + Key Lookup)

مثال:

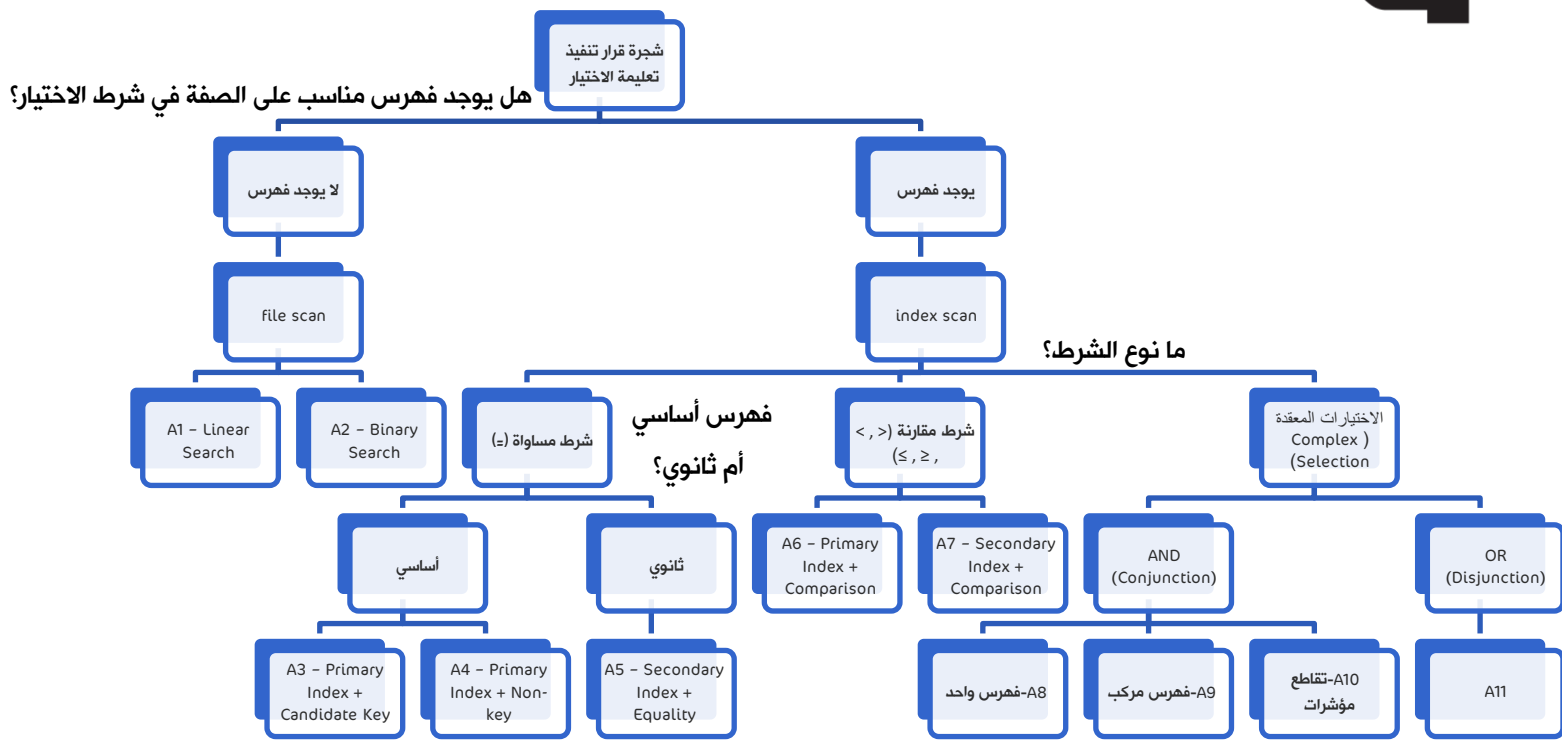
NOT (branch_name = 'Damascus')

الخلاصة:

نفس الاستعلام قد يُنفَّذ بعشرات الطرق وال DBMS يختار الطريقة ذات أقل كلفة.

لا تختظر اللحظة المناسبة

اصنعها بالالتزام اليومي



بعد أن تعرّفنا كيف ينفّذ النظام تعليمية الاختيار، وكيف يصل إلى السجلات التي تحقق شرطاً معيناً بأقل كلفة ممكنة، ننتقل الآن إلى عملية أخرى شائعة جداً في قواعد المعطيات، وهي **الفرز (Sorting)**.

كثير من الاستعلامات لا تكتفي بإيجاد السجلات، بل تطلب عرضها مرتبة، مثل:

```
SELECT * FROM Account
ORDER BY balance;
```

لذلك، من الضروري أن نفهم كيف ينفّذ النظام عملية الفرز، وما هي الخوارزميات المستخدمة، ومتى نستخدم كل واحدة منها.

قد يخطر في بالنا تنفيذ الفرز ببساطة عن طريق **بناء فهرس مرتب (Ordered Index)** على الصفة المطلوبة، ثم قراءة السجلات وفق ترتيب الفهرس.

لكن المشكلة أن الفهرس لا يضمن أن السجلات نفسها مخزنة بترتيب متتال على القرص، وبالتالي:

- كل سجل قد يكون في كتلة مختلفة
- أي أننا قد نضطر إلى قراءة كتلة قرص لكل سجل

وهذا ما يُسمّى: Random I/O

وهو بطيء جداً مقارنة بالقراءة التسلسلية.

لذلك، الفرز باستخدام الفهرس غير عملي عند التعامل مع عدد كبير من السجلات.

بناءً على ذلك، يلجأ نظام إدارة قواعد المعطيات إلى **خوارزميات فرز خاصة**، تختلف حسب:

- هل العلاقة يمكن تحميلها كاملة إلى الذاكرة؟
- أم أن حجمها أكبر من الذاكرة المتاحة؟

1. الحالة الأولى: العلاقة تدخل كاملة في الذاكرة

إذا كانت العلاقة صغيرة ويمكن تحميلها بالكامل في الذاكرة الرئيسية، يمكن استخدام خوارزميات الفرز الداخلية مثل:

- Quicksort
- Heapsort

2. الحالة الثانية: العلاقة أكبر من الذاكرة

أما إذا كانت العلاقة كبيرة ولا يمكن تحميلها كاملة إلى الذاكرة، فنستخدم:

- External Sort-Merge

وهي الخوارزمية الأساسية التي يعتمد عليها ال DBMS في الفرز الخارجي.

✚ فكرة عامة قبل الدخول في التفاصيل:

الفكرة الأساسية:

- بما أن الذاكرة محدودة
- نقوم بفرز أجزاء صغيرة من العلاقة
- ثم ندمج هذه الأجزاء المفروزة معاً
- إلى أن نحصل على العلاقة كاملة بشكل مفروز

✚ الذاكرة المتاحة (M):

نفترض أن:

(حجم الذاكرة المتاحة لعملية الفرز) M = عدد الكتل أو الصفحات التي يمكن تحميلها في الذاكرة في نفس الوقت
هذا العدد **محدود**، وهو السبب الأساسي لاستخدام الفرز الخارجي.

✚ لماذا نحتاج إلى عدة أطوار (Passes)؟

بسبب محدودية الذاكرة:

- لا يمكن فرز العلاقة كاملة مرة واحدة
- لذلك نحتاج إلى عدة أطوار (Passes)

وكل طور يتألف من مرحلتين أساسيتين:

1. مرحلة التجزئة (Create Sorted Runs):

ماذا تعني هذه المرحلة؟

نقوم بتقسيم العلاقة الكبيرة إلى **علاقات جزئية صغيرة**، بحيث يمكن تحميل كل علاقة جزئية بالكامل إلى الذاكرة وفرزها.

كيف تتم العملية عملياً؟

نكرر الخطوات التالية حتى نهاية العلاقة:

1. نقرأ M كتل من العلاقة من القرص
2. نحمل هذه الكتل إلى الذاكرة.
3. نُجري الفرز داخل الذاكرة (باستخدام خوارزمية داخلية)

4. نكتب نتيجة المفروزة على القرص على شكل علاقة جزئية تسمى: Run ونسميها: $R_1, R_2, R_3, \dots$ ، ثم: $i = i + 1$

نتيجة هذه المرحلة:

بعد الانتهاء نحصل على:

■ N علاقات جزئية مفروزة (Runs)

حيث:

■ $N =$ عدد المسارات الناتجة من مرحلة التجزئة.

ملاحظة:

إلى هنا نكون وصلنا إلى مرحلة فيها:
كل Run مفروز داخليا، لكن العلاقة ككل لم تُفرز بعد.

2. مرحلة الدمج (Merge Phase):

الهدف الآن:

■ دمج العلاقات الجزئية المفروزة للحصول على علاقة واحدة مفروزة.

شرط مهم:

نفترض $N < M$ ، أي:

■ لدينا ذاكرة كافية لدمج عدة Runs في نفس الوقت.

كيف يتم الدمج؟

نستخدم أسلوب (N-way merge)، استخدام الذاكرة أثناء الدمج:

نستخدم الذاكرة كما يلي:

■ N كتل ل Input Buffers كتلة واحدة من كل Run

■ كتلة واحدة ك Output Buffer

خطوات الدمج (تتكرر):

نكرر الخطوات التالية:

1. نختار أصغر سجل (حسب ترتيب الفرز) من بين جميع صفحات الإدخال

2. نكتب هذا السجل في Output Buffer

3. إذا امتلأ Output Buffer:

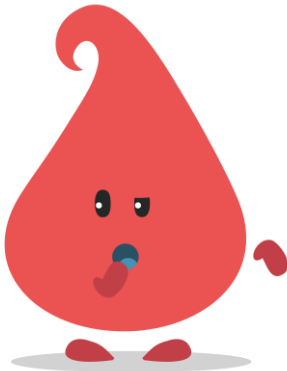
■ نكتبه على القرص

4. نحذف السجل من صفحة الإدخال التي أخذ منها

5. إذا أصبحت صفحة الإدخال فارغة:

■ نقرأ الكتلة التالية من نفس ال Run إلى الذاكرة.

نستمر: حتى تصبح جميع صفحات الإدخال فارغة.



هل نحتاج لأكثر من طور دمج؟

نعم، في حال كان:

- عدد ال Runs أكبر من $M - 1$

لأن:

- في كل طور دمج يمكننا دمج $M - 1$ علاقة جزئية فقط.

وبالتالي:

- في كل طور يقل عدد العلاقات الجزئية
- وقد نحتاج إلى عدة أطوار دمج

مثال تطبيقي:

لدينا السجلات التالية (غير مفروزة):

المعطيات:

- عدد الكتل $br = 12$
- حجم الذاكرة $M = 3$ كتل
- كل كتلة تحتوي سجلاً واحداً

مرحلة التجزئة، في كل مرة:

- نقرأ 3 كتل
- نفرزها
- نكتب Run مفروز

مثال:

Run₁: (a24, d31, g24)

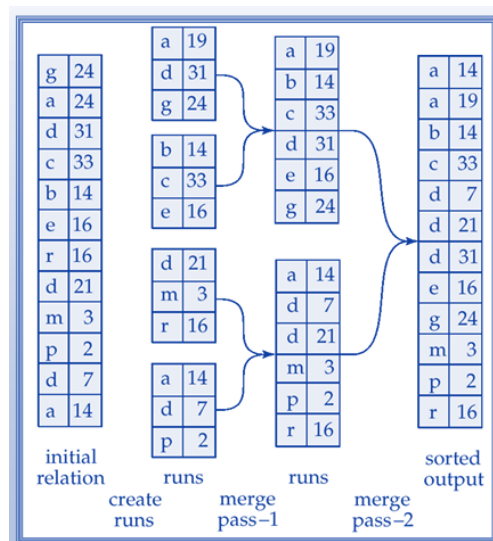
Run₂: (b14, c33, e16)

مرحلة الدمج:

- ندمج Runs باستخدام N-way merge
- حتى نحصل على ملف واحد مفروز بالكامل

الشكل التالي يوضح الآلية:

g	24
a	24
d	31
c	33
b	14
e	16
r	21
d	21
m	3
p	2
d	7
a	14



تحليل الكلفة (Cost Analysis):

عدد الأطوار:

عدد الأطوار المطلوبة للدمج هي: $\lceil \log_{M-1}(br/M) \rceil$

كلفة كل طور:

في كل طور (عدا الأخير):

- نقرأ كل كتل العلاقة.
- ثم نكتبها من جديد

أي:

- $2 \times br$ عمليات وصول للقرص

ملاحظة مهمة:

في الطور الأخير:

لا نكتب النتيجة على القرص، بل يمكن إرسالها مباشرة للاستعلام لذلك:

لا تُحسب كلفة الكتابة في الطور الأخير

الكلفة الكلية للفرز الخارجي:

عدد عمليات الوصول للقرص هو: $br(2\lceil \log_{M-1}(br/M) \rceil + 1)$

باختصار External Sort-Merge يقوم بفرز علاقة كبيرة عبر تقسيمها إلى أجزاء مفروزة، ثم دمجها على عدة أطوار، مع تقليل عدد عمليات الوصول العشوائي للقرص.

تناولت هذه المحاضرة تنفيذ عمليات الاختيار والفرز في DBMS مع تحليل الكلفة المتوقعة لكل طريقة تنفيذ.

في المحاضرة القادمة سنكمل دراسة تنفيذ الاستعلامات بالتركيز على عمليات الدمج بين العلاقات.

The End...

النجاح الحقيقي يبدأ حين تواصل رغم غياب الحماس لا بسببه

